

GraphRAG

From Local to Global: Knowledge Graph-Guided Retrieval

Edge et al. (2024) | Microsoft Research | arXiv:2404.16130

What is GraphRAG?

GraphRAG (Edge et al., Microsoft Research, 2024) extends standard RAG by building a knowledge graph from the source documents and using both graph community summaries and vector similarity search to answer queries. It dramatically improves performance on global queries that require synthesizing information across an entire corpus.

Source: Edge, D. et al. (2024). From Local to Global: A Graph RAG Approach. arXiv:2404.16130

1. Why GraphRAG?

Standard RAG retrieves semantically similar chunks but fails at questions requiring global synthesis: 'What are the main themes in these 500 documents?' — no single chunk contains this answer. GraphRAG addresses this with community detection and hierarchical summarization at graph level.

2. GraphRAG Pipeline

- 1. Text Chunking:** Split documents into overlapping chunks (~600 tokens)
- 2. Entity Extraction:** LLM extracts entities (people, places, concepts) and relationships from each chunk
- 3. Graph Construction:** Build a property graph: nodes = entities, edges = relationships
- 4. Community Detection:** Leiden algorithm groups nodes into hierarchical communities
- 5. Community Summarization:** LLM generates summaries for each community at each level
- 6. Global Search:** Map-reduce: generate partial answers for each community, reduce to final answer
- 7. Local Search:** Standard vector search + entity subgraph expansion for specific queries

3. Implementation with Microsoft GraphRAG

```
# Install: pip install graphrag
# Initialize: python -m graphrag init --root ./my_data

# settings.yml key configurations:
# llm: { model: claude-opus-4-5, api_base: ... }
```

```

# embeddings: { model: text-embedding-3-large }
# chunks: { size: 600, overlap: 100 }
# community_reports: { max_length: 2000 }

# Index the corpus
python -m graphrag index --root ./my_data

# Query globally (synthesizes across entire corpus)
python -m graphrag query \
  --root ./my_data \
  --method global \
  --query "What are the main themes in this document set?"

# Query locally (specific entities)
python -m graphrag query \
  --root ./my_data \
  --method local \
  --query "How does RAG compare to fine-tuning?"

# Output: Parquet files with entity, relationship, and community tables
# entities.parquet: node metadata
# relationships.parquet: edge data
# community_reports.parquet: multi-level summaries

```

4. Performance Comparison

Query Type	Standard RAG	GraphRAG
Specific fact lookup	Excellent	Good (higher latency)
Multi-hop reasoning	Poor	Excellent
Global thematic summary	Very poor	Excellent
Entity relationship queries	Poor	Excellent
Cost per query	Low	High (many LLM calls)
Index build time	Minutes	Hours (entity extraction)